

Universidad de Caldas
Facultad de Inteligencia Artificial e Ingenierías
Análisis y Diseño de Algoritmos — 2026-1

Proyecto K-QGMIP

Manual Técnico

Partición de Mínima Información en k -particiones
Estrategias **KGeoMIP** (geométrica) y **KQNodes** (submodular)

Teoría de la Información Integrada (IIT)
 $k \in \{2, 3, 4, 5\}$ — n hasta ≈ 25 nodos

Autores

Milton Pablo Arias Rincon - 38790
Nelson Daniel Estrada Leiton - 9665

junio de 2026

Índice

1. Resumen ejecutivo	2
1.1. Problema abordado	2
1.2. Enfoque algorítmico	2
1.3. Principales resultados	2
1.4. Alcance y recomendaciones de uso	2
2. Fundamentos teóricos	3
2.1. Sistema, subsistema y repertorio	3
2.2. Definición formal de k -partición	3
2.3. Función objetivo: la pérdida δ_k	3
2.4. Extensión del marco GeoMIP/QNodes a k -particiones	4
2.5. Complejidad del espacio de soluciones	4
3. Arquitectura del software	4
3.1. Diagrama de clases	5
3.2. Diagrama de paquetes	6
3.3. Diagrama de secuencia	6
3.4. Patrones de diseño aplicados	7
3.5. Decisiones arquitectónicas clave	7
4. Diseño algorítmico	7
4.1. Visión general	7
4.2. KGeoMIP: generación geométrica del <i>cut pool</i>	8
4.3. KQNodes: generación submodular del <i>cut pool</i>	9
4.4. Estructuras de datos	9
4.5. Estrategia de búsqueda	10
4.6. Evaluación de particiones: marginal local	10
4.7. Reutilización entre valores de k	10
4.8. Optimizaciones implementadas	10
4.9. Metaheurísticas comparativas (GA / SA / Tabú)	11
5. Análisis de complejidad	11
6. Detalles de implementación	12
6.1. Métodos principales	12
6.2. Dependencias externas	13
6.3. Ingeniería de software	13
6.4. Tests implementados	13
7. Resultados experimentales	14
7.1. Datasets	14
7.2. Métricas	14
7.3. Tabla de resultados	14
7.4. Gráficas y visualizaciones	15
7.5. Visualización interactiva e interfaz web	16
7.6. Análisis de resultados	16
7.7. Interpretación de los resultados	17

7.8. Validación de correctitud	18
7.8.1. Frente al óptimo exacto y al legado $k = 2$	18
7.8.2. Frente al proyecto original de la docente	18
7.8.3. Re-evaluación de las particiones almacenadas	18
8. Limitaciones y trabajo futuro	18
8.1. Limitaciones conocidas	18
8.2. Supuestos y restricciones	19
8.3. Mejoras potenciales y trabajo futuro	19
9. Apéndices técnicos	19
9.1. Reducción $k = 2 \Rightarrow$ bipartición legada	19
9.2. Reproducción de figuras y diagramas	19
9.3. Uso de inteligencia artificial generativa	19
9.4. Referencias	20

1. Resumen ejecutivo

1.1. Problema abordado

La **Teoría de la Información Integrada** (IIT) [1, 2] cuantifica cuánta información genera un sistema *como un todo* por encima de la suma de sus partes. El indicador central es Φ , que mide la pérdida de información al *cortar* el sistema según su **Partición de Mínima Información** (MIP): la división que menos información destruye. El sistema se describe mediante una **Matriz de Probabilidad de Transición** (TPM) sobre variables binarias.

El marco de referencia (GeoMIP y QNodes) resuelve únicamente **biparticiones** ($k = 2$). Este proyecto lo **extiende a k -particiones** con $k \in \{2, 3, 4, 5\}$ y escala el tamaño del sistema hasta $n \approx 25$ nodos. Esta generalización es relevante porque la verdadera estructura de un sistema rara vez es binaria: una partición en k bloques captura modularidad de grano más fino que un único corte.

1.2. Enfoque algorítmico

Se implementan dos estrategias núcleo, ambas heredan de la clase abstracta SIA y se evalúan con la *misma* pérdida δ_k :

- **KGeoMIP** (geométrica): interpreta el espacio de estados como un hipercubo, construye *una sola vez* la tabla de costos de transición T y aplica refinamiento jerárquico voraz ($k - 1$ biparticiones sucesivas).
- **KQNodes** (submodular): obtiene candidatos mediante una búsqueda tipo Queyranne y los refina con el mismo motor voraz que KGeoMIP.

Como apoyo se incluyen un algoritmo **exacto** (enumeración de Stirling, referencia para n pequeño) y una **línea base** de *clustering* espectral/KMeans determinista [7].

1.3. Principales resultados

- **Correctitud:** para $k = 2$, KGeoMIP y KQNodes reproducen *exactamente* GeoMIP y QNodes (invariante de regresión). KGeoMIP coincide con el óptimo exacto en los casos golden y KQNodes acierta 9/10 frente al oráculo.
- **Calidad:** las dos estrategias núcleo obtienen pérdidas δ_k *órdenes de magnitud menores* que la línea base de clustering (p. ej. 0,47 vs. 4,69 en N10A, $k = 2$).
- **Eficiencia:** tabla de costos vectorizada e indexada por el entero del estado ($\sim 20\times$ a $m=10$ y $\sim 80\times$ a $m=15$ sobre la versión con diccionario, creciente con m) y marginal local; tensores float32 (pico medido 8,34 GB a $n = 25$, cabe en 16 GB) y paralelismo por procesos (PCD).

1.4. Alcance y recomendaciones de uso

Ambas estrategias núcleo resuelven el subsistema completo de N25A con $k \in \{2, 3, 4, 5\}$ en la máquina de referencia (Intel Core i7-10510U, 16 GB): KQNodes en ≈ 26 s y KGeoMIP en ≈ 101 s para los cuatro valores de k (la preparación se comparte entre ellos). Esto fue posible tras dos cambios de implementación que no alteran la fundamentación teórica (permitidos explícitamente por la especificación §3.2): la tabla de costos pasó de un diccionario de tuplas a un arreglo indexado por el entero del estado, y la evaluación de δ_k pasó de marginalizar el tensor completo a un promedio local sobre el bloque que de verdad se necesita. La recomendación operativa es ejecutar *ambas* estrategias núcleo y tomar el mínimo: comparten motor voraz y métrica, así que su δ_k coincide en la práctica (y su tasa de acierto exacto en $k \geq 3$ es equivalente). KGeoMIP

es más rápida en n pequeño y KQNodes en n grande ($n \geq 22$); el clustering queda solo como referencia exploratoria.

2. Fundamentos teóricos

2.1. Sistema, subsistema y repertorio

Un sistema de n variables binarias se describe por una TPM determinista $T \in \{0,1\}^{2^n \times n}$ en orden *little-endian* (fila 0 = 00...0). Internamente cada columna se representa como un NCube: un tensor de forma $(2, \dots, 2)$ (n ejes). El subsistema bajo análisis se obtiene fijando condiciones de fondo (*condition*) y restando *purview/mechanism* (*subtract*). Su distribución marginal de efecto P es el repertorio que se desea preservar.

2.2. Definición formal de k -partición

Sea el subsistema con un *universo futuro* (*purview*) F de índices de efecto en $t+1$ y un *universo presente* (*mechanism*) M de índices de mecanismo en t . Una k -**partición** es una colección de k bloques

$$\mathcal{P} = \{(F_1, M_1), (F_2, M_2), \dots, (F_k, M_k)\}$$

tal que las partes *futuras* forman una partición de F y las partes *presentes* una partición de M :

$$\bigcup_{j=1}^k F_j = F, \quad F_i \cap F_j = \emptyset \ (i \neq j); \quad \bigcup_{j=1}^k M_j = M, \quad M_i \cap M_j = \emptyset \ (i \neq j).$$

Semántica estricta. Se adopta la definición **estricta** (doc. oficial §2.1 [11]): los k bloques deben ser *no vacuos* (cada bloque tiene al menos un átomo, futuro o presente). Un *átomo* es un par (tiempo, índice) con tiempo $\in \{\text{ACTUAL}, \text{EFECTO}\}$. La validación se impone en `KPartition.__post_init__`: si algún bloque queda totalmente vacío se lanza `ValueError`. Esto evita la degeneración en la que $k > 2$ colapsa a una bipartición (un bloque vacío sería “inerte”), garantizando que la tabla experimental sea genuina y la pérdida *monótona* en k .

Ejemplo ($n = 3$). Con $F = M = \{0, 1, 2\}$, una 3-partición válida es

$$\mathcal{P} = \{(\{0\}, \{0\}), (\{1\}, \{1, 2\}), (\{2\}, \emptyset)\}.$$

El bloque 3 es no vacuo (tiene el átomo de efecto 2) aunque su parte presente sea vacía; los tres bloques cubren F y M de forma disjunta.

2.3. Función objetivo: la pérdida δ_k

Cada k -partición induce una *reconstrucción factorizada* del subsistema: los bloques se evalúan de forma independiente y se recombinan por producto tensorial. La pérdida de información es la distancia *Earth Mover's Distance* (EMD) entre el repertorio original P y el reconstruido $P_{\mathcal{P}}$:

$$\delta_k(\mathcal{P}) = \text{EMD}(P, P_{\mathcal{P}}). \quad (1)$$

Bajo independencia condicional del repertorio de efecto, la EMD [6] se reduce a la suma de diferencias absolutas componente a componente (EMD-efecto *analítica*, coste $\mathcal{O}(2^{|F|})$ sin matriz de costos):

$$\text{EMD}(u, v) = \sum_i |u_i - v_i|. \quad (2)$$

El problema de optimización es entonces

$$\mathcal{P}^* = \arg \min_{\mathcal{P} \in \Pi_k(F, M)} \delta_k(\mathcal{P}), \quad (3)$$

donde $\Pi_k(F, M)$ es el conjunto de k -particiones estrictas. La k -MIP es $\mathcal{P}^*$ y su valor $\delta_k(\mathcal{P}^*)$ generaliza a Φ .

2.4. Extensión del marco GeoMIP/QNodes a k -particiones

La clave de la extensión (doc. §2.2) es que **una k -partición equivale a $k - 1$ biparticiones sucesivas**: dividir el hipercubo con $k - 1$ hiperplanos es lo mismo que cortar un bloque en dos, repetidamente. Esto permite *reutilizar* la maquinaria de bipartición de ambas estrategias:

- GeoMIP propone cortes geométricos desde la tabla T ; KGeoMIP los proyecta sobre subbloques y elige, en cada paso, el corte que minimiza (1).
- QNodes propone cortes submodulares (Queyranne); KQNodes los refina con el mismo motor voraz.

Para $k = 2$ el refinamiento ejecuta *un* corte y por tanto recupera la bipartición legada exactamente (invariante 1). La justificación de calidad es doble: (i) cada corte hereda la garantía de su método base (GeoMIP es óptimo exacto en biparticiones sobre el núcleo verificado; Queyranne [4] es exacto para funciones submodulares simétricas [5] en $\mathcal{O}(n^3)$), y (ii) la elección voraz guiada por δ_k asegura *monotonía* y validación contra el exacto en n pequeño. El óptimo global de k -particiones *no* se garantiza (el voraz es heurístico), lo que se documenta honestamente en §8.

2.5. Complejidad del espacio de soluciones

El número de particiones de un conjunto de a átomos en exactamente k bloques no vacíos es el **número de Stirling de segunda especie** $S(a, k)$. Para el subsistema completo $a = |F| + |M| = 2n$, de modo que el espacio crece como

$$|\Pi_k| = S(2n, k) \sim \frac{k^{2n}}{k!} \quad (n \text{ grande}).$$

Comparado con biparticiones, donde $S(2n, 2) = 2^{2n-1} - 1$ es ya exponencial, el caso k -partita es *super-exponencial* en k . La enumeración exacta (Stirling) solo es viable para $n \lesssim 6$; de ahí la necesidad de las heurísticas KGeoMIP/KQNodes, que recorren un subconjunto dirigido de tamaño polinómico de candidatos.

3. Arquitectura del software

El sistema sigue el patrón **Strategy + Template Method** sobre la clase abstracta SIA. La Figura 1 muestra los componentes principales y su integración.

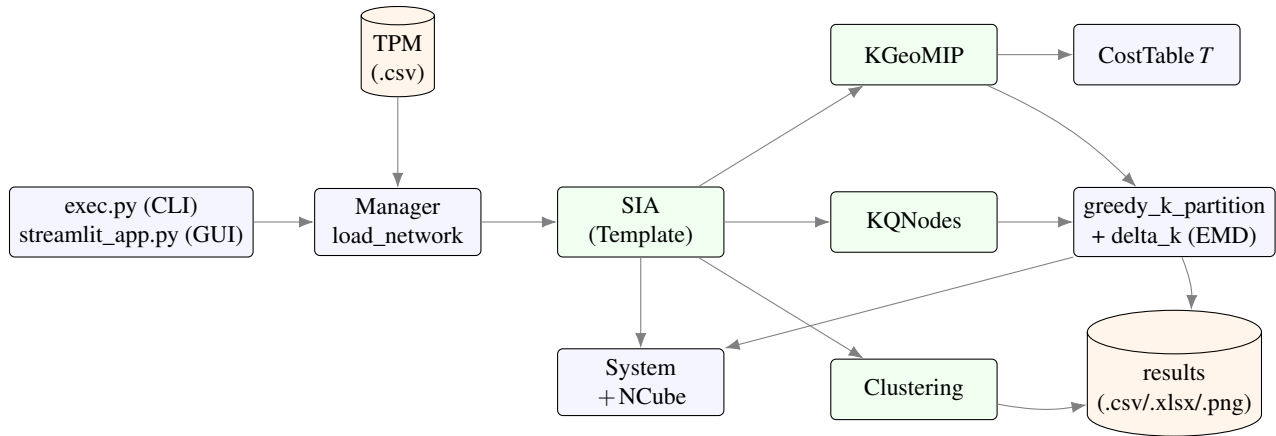


Figura 1: Arquitectura general: el Manager carga la TPM, cada estrategia hereda de SIA, comparte el motor `greedy_k_partition` y puntúa con `delta_k`.

3.1. Diagrama de clases

La Figura 2 resume la jerarquía de herencia y las relaciones de composición. La versión editable (PlantUML) está en `docs/manuales/diagrams/02_clases.puml`.

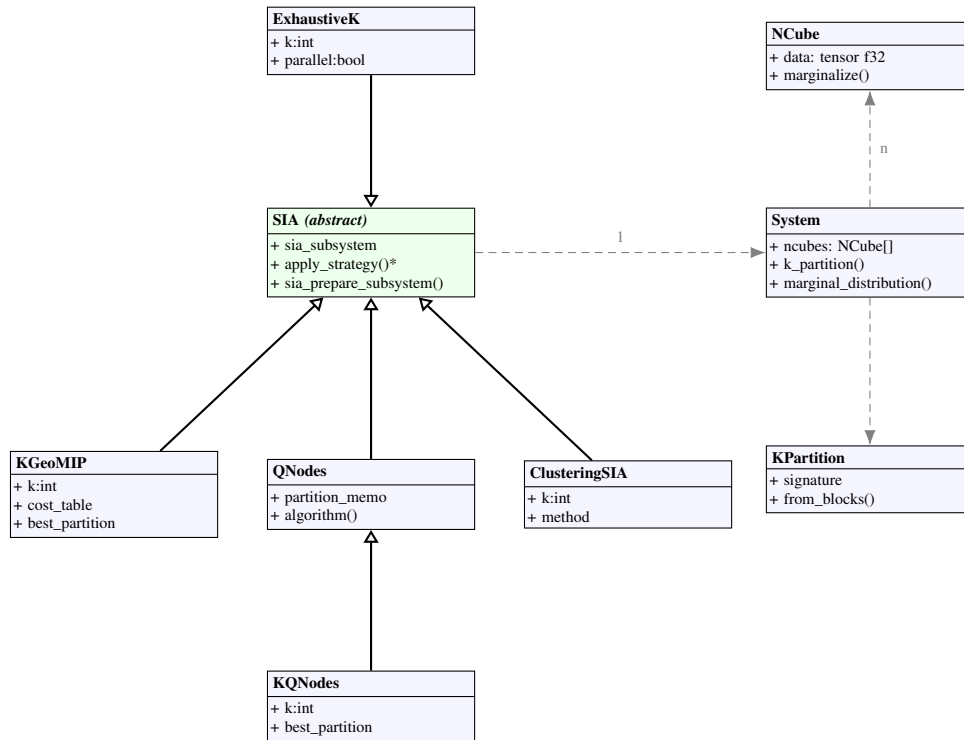


Figura 2: Diagrama de clases (núcleo). Flechas huecas: herencia; discontinuas: composición/dependencia. `KGeoMIP` y `KQNodes` producen una `KPartition` puntuada vía `System.k_partition`.

3.2. Diagrama de paquetes

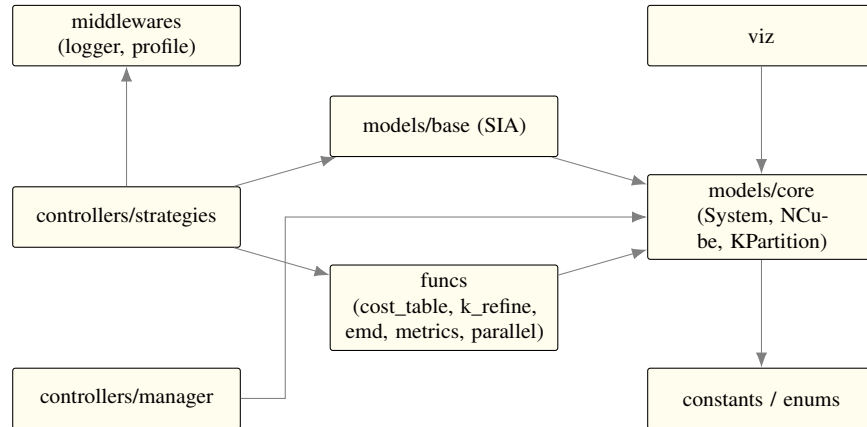


Figura 3: Organización modular en `src/`. Las estrategias dependen del modelo base, del núcleo de dominio y de `funcs`.

3.3. Diagrama de secuencia

La Figura 4 traza la búsqueda de la k -MIP con KGeoMIP.

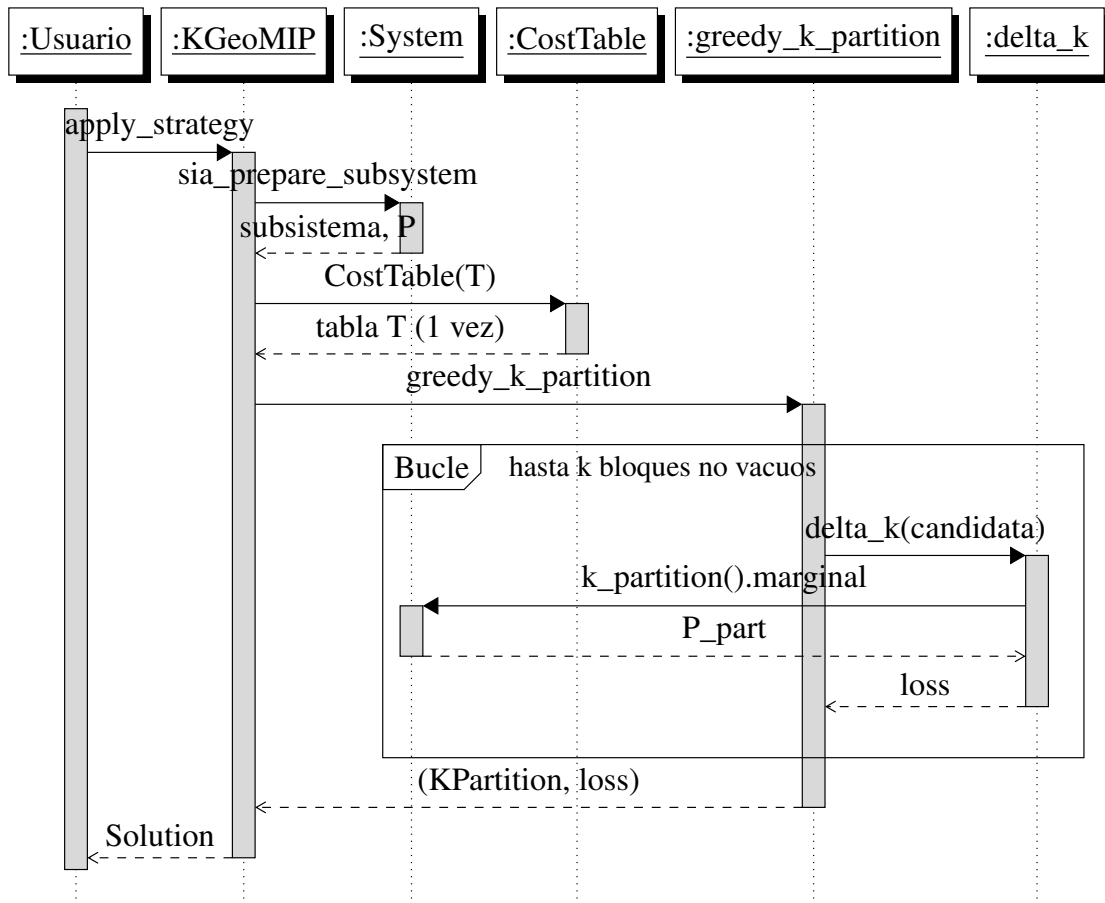


Figura 4: Secuencia de la búsqueda k -partita (KGeoMIP). La tabla T se construye una sola vez; el bucle `for` puntúa cada candidata con `delta_k`.

3.4. Patrones de diseño aplicados

- **Strategy:** cada algoritmo (KGeoMIP, KQNodes, Clustering, ExhaustiveK) es una estrategia intercambiable con interfaz común `apply_strategy`.
- **Template Method:** `SIA.sia_prepare_subsystem` fija el esqueleto compartido (System $\rightarrow$ condition $\rightarrow$ subtract $\rightarrow$ marginal); las subclases solo implementan la búsqueda.
- **Singleton:** `application` centraliza la configuración global (semilla, página de red, métrica, notación, EMD, profiling).
- **Objeto inmutable / value object:** `KPartition` se autovalida en construcción (cobertura, disyunción, k bloques no vacuos).

3.5. Decisiones arquitectónicas clave

- **Reutilización del corte base:** en lugar de reimplementar GeoMIP y QNodes, sus pools de candidatos se *proyectan* sobre subbloques. Garantiza el invariante $k = 2$ y reduce la deuda técnica.
- **Motor voraz compartido** (`greedy_k_partition`): un único punto de verdad para la construcción k -partita; ambas estrategias difieren solo en *cómo* generan el *cut pool*.
- **Tabla T una sola vez:** la estructura cara se calcula una vez por sistema y se reutiliza para todo k (doc. §3).
- `float32`: se sacrifica precisión irrelevante (validada a 10^{-5}) por la mitad de memoria, lo que habilita $n = 25$ en RAM.

4. Diseño algorítmico

4.1. Visión general

Ambas estrategias comparten la filosofía “*divide y refina*”: parten del subsistema como un único bloque y lo dividen $k - 1$ veces, eligiendo cada vez el corte que minimiza δ_k . Solo difieren en el origen de los cortes candidatos (geométrico vs. submodular). El Algoritmo 1 es el motor común.

Algoritmo 1 GREEDYKPARTITION — motor voraz compartido**Require:** subsistema S , repertorio base P , *cut pool* C , universos F, M , entero k **Ensure:** k -partición $\mathcal{P}$ con su pérdida δ_k

```

1:  $blocks \leftarrow [(F, M)]$  ▷ un solo bloque inicial
2: while  $|blocks| < k$  do
3:    $best \leftarrow \infty$ ;  $bestBlocks \leftarrow \text{NONE}$ 
4:   for cada bloque  $b = (F_b, M_b)$  en  $blocks$  con  $|F_b| + |M_b| \geq 2$  do
5:     for cada corte  $c = (F_c, M_c)$  en  $C$  do
6:        $in \leftarrow (F_b \cap F_c, M_b \cap M_c)$ ;  $out \leftarrow (F_b \setminus F_c, M_b \setminus M_c)$ 
7:       if  $in$  vacuo o  $out$  vacuo then continue
8:       end if
9:        $\mathcal{P}' \leftarrow$  reemplazar  $b$  por  $\{in, out\}$  en  $blocks$ 
10:       $\ell \leftarrow \delta_k(S, \mathcal{P}', P)$  ▷ Ec. (1)
11:      if  $\ell < best$  then  $best \leftarrow \ell$ ;  $bestBlocks \leftarrow \mathcal{P}'$ 
12:      end if
13:    end for
14:  end for
15:  if  $bestBlocks = \text{NONE}$  then error “no refinable a  $k$  bloques”
16:  end if
17:   $blocks \leftarrow bestBlocks$ 
18: end while
19: return (KPARTITION( $blocks$ ),  $\delta_k(S, blocks, P)$ )

```

4.2. KGeoMIP: generación geométrica del *cut pool*

KGeoMIP construye la tabla T (Algoritmo 2) y deriva de ella los cortes candidatos. T recorre el hipercubo por niveles de Hamming desde el estado inicial hacia su antípoda, almacenando un vector de costo por nodo, ponderado por $\gamma = 2^{-d_H}$.

Algoritmo 2 COSTTABLE — tabla de costos T , vectorizada por niveles**Require:** arreglos planos por nodo X_i , estado inicial s_0 , m dimensiones

```

1:  $T \leftarrow$  arreglo  $(2^m, n)$  en float32;  $T[s_0] \leftarrow 0$ 
2:  $dist[s] \leftarrow d_H(s_0, s)$  para todo  $s$  ▷ popcount vectorizado, una vez
3: for  $d = 1 \dots m$  do ▷ niveles de Hamming en orden ascendente
4:    $L \leftarrow \{s : dist[s] = d\}$ , procesado en chunks de  $2^{20}$  filas
5:    $D[s, i] \leftarrow |X_i[s] - X_i[s_0]|$  para  $s \in L$  ▷ gather por columna
6:   if  $d > 1$  then
7:     for cada posición de bit  $j$  do  $D[s] += T[s \oplus 2^j]$  si el bit  $j$  de  $s \oplus s_0$  está activo
8:     end for
9:   end if
10:   $T[L] \leftarrow 2^{-d} \cdot D$ 
11: end for
12: return  $T$ , candidatos =  $n$  cortes de nodo único + mejor corte por nivel

```

La recurrencia es la misma del recorrido BFS original (cada nivel acumula los costos de sus vecinos un bit más cerca del origen), pero el estado se indexa por su entero *little-endian* en un arreglo contiguo en lugar de un diccionario de tuplas de Python. La diferencia no es cosmética: a $m = 25$ el diccionario necesita unas

2^{25} entradas con un costo de objeto de ~ 1 KB cada una (decenas de GB, irrealizable), mientras que el arreglo ocupa 3,36 GB y se construye en unos dos minutos. La igualdad con la versión original es bit a bit, incluido el orden de desempate de los candidatos (el recorrido BFS visita los niveles en orden lexicográfico de las posiciones volteadas, que el código vectorizado reproduce con máscaras bit-invertidas); la clase original se conserva como `LegacyCostTable` y un conjunto de 38 pruebas verifica la igualdad de tabla, candidatos y consultas sobre TPMs aleatorias y deterministas.

4.3. KQNodes: generación submodular del *cut pool*

KQNodes ejecuta una vez la búsqueda tipo Queyranne heredada de QNodes para poblar `partition_memo`. Cada clave del memo representa un lado de una bipartición candidata como colección de vértices (tiempo, índice); se aplanan y se separa por capa en un corte (F_c, M_c) . Para $k = 2$, el refinamiento aplica un único corte y reproduce QNodes exactamente (incluidos sus casos subóptimos conocidos, congelados en los tests de triaje). El Algoritmo 3 resume la generación del *cut pool*; el refinamiento posterior es el mismo GREEDYKPARTITION (Algoritmo 1).

Algoritmo 3 KQNODESCUTPOOL — *cut pool* submodular (Queyranne)

Require: subsistema S , vértices $V = \{(\text{ACTUAL}, i)\} \cup \{(\text{EFECTO}, i)\}$

Ensure: *cut pool* C de cortes (F_c, M_c)

```

1:  $memo \leftarrow \emptyset$  ▷ candidatos de bipartición (claves de partition_memo)
2: while  $|V| > 1$  do ▷ secuencia tipo Queyranne, [4]
3:   elegir un vértice inicial  $v_0$ ;  $\Omega \leftarrow \{v_0\}$ ;  $\Delta \leftarrow V \setminus \{v_0\}$ 
4:   while  $|\Delta| > 1$  do
5:      $u^* \leftarrow \arg \min_{u \in \Delta} \delta_k(\Omega \cup \{u\}) - \delta_k(\{u\})$  ▷ ganancia marginal submodular
6:      $\Omega \leftarrow \Omega \cup \{u^*\}$ ;  $\Delta \leftarrow \Delta \setminus \{u^*\}$ 
7:   end while
8:   añadir el par candidato (último  $\Omega$ , último  $\Delta$ ) a memo
9:   fusionar el par de menor  $\delta_k$  en un solo vértice (contracción de Queyranne)
10: end while
11:  $C \leftarrow \emptyset$ 
12: for cada lado candidato  $s$  en memo do
13:   aplanar  $s$  en vértices (tiempo, índice)
14:    $F_c \leftarrow \{i : (\text{EFECTO}, i) \in s\}$ ;  $M_c \leftarrow \{i : (\text{ACTUAL}, i) \in s\}$ 
15:    $C \leftarrow C \cup \{(F_c, M_c)\}$ 
16: end for
17: return  $C$  ▷ insumo de GREEDYKPARTITION (Algoritmo 1)

```

4.4. Estructuras de datos

- **NCube** — tensor $(2, \dots, 2)$ float32 por columna de la TPM; `marginalize` es puro (devuelve un nuevo NCube) y memoizado.
- **System** — colección de NCubes con `condition`, `subtract`, `bipartition` (memoizada), `k_partition` y `marginal_distribution`.
- **CostTable** — arreglo contiguo $(2^m, n)$ en float32 indexado por el entero del estado; el costo de la transición $s_0 \rightarrow s'$ es la fila $T[\text{int}(s')]$. La construcción procesa los niveles de Hamming en *chunks* acotados (constante `COST_TABLE_CHUNK_ROWS`) y cachea el popcount para reutilizarlo al enumerar candidatos.

- **KPartition** — bloques pareados (F_j, M_j) como tuplas ordenadas; *signature* es la representación canónica usada para puntuar y comparar.
- **Block / cut** — par frozenset (F, M) usado en el motor voraz; dividir b con c da $(b \cap c, b \setminus c)$.

4.5. Estrategia de búsqueda

La técnica de diseño es **voraz jerárquica** (greedy): en lugar de enumerar $S(2n, k)$ particiones, se realizan $k - 1$ decisiones locales óptimas. El *cut pool* acota el ramaje a $\mathcal{O}(n)$ cortes por nivel. Se trata de una heurística: garantiza factibilidad y monotonía pero no optimalidad global; por eso se valida contra el exacto (§7).

4.6. Evaluación de particiones: marginal local

Una candidata se puntúa con `delta_k` (Ec. (1)). La versión original construía el subsistema particionado completo (`System.k_partition`) y lo marginalizaba, con costo $\mathcal{O}(2^n)$ por candidata. La observación clave es que la EMD-efecto solo necesita *un escalar por nodo*: la probabilidad marginal evaluada en el estado inicial. Ese escalar se obtiene fijando primero las dimensiones conservadas al bit del estado inicial (una vista del tensor, sin copia) y promediando únicamente el bloque restante:

$$P_i = \text{mean}(X_i[b_{j_1}, \dots, b_{j_r}, :, \dots, :]), \quad \text{costo } \mathcal{O}(2^{\text{dims descartadas}}).$$

`NCube.marginal_value` implementa este promedio local con memo por conjunto de ejes, y `System` expone `bipartition_marginal_distribution` y `k_partition_marginal_distribution` con la misma semántica de ejes que las operaciones originales. Para las TPMs deterministas del proyecto el resultado es idéntico bit a bit (las medias son diádicas); para datos `float32` arbitrarios la diferencia medida es a lo sumo 1 ulp ($1,9 \cdot 10^{-7}$), por el orden de la suma por pares de NumPy. El método original se conserva y los tests comparan ambas vías. El efecto práctico: QNodes sobre el subsistema completo de N20A pasó de cientos de segundos (con la marginalización completa) a $\approx 1,6$ s con la marginal local.

4.7. Reutilización entre valores de k

La especificación exige que la tabla de costos se calcule una vez por sistema y se reutilice para todo k (§3, línea 103 del documento oficial). `apply_strategy_for_ks(condition, purview, mechanism, ks)` materializa ese contrato: la preparación cara (subsistema, tabla T o secuencia de Queyranne, *cut pool*) se ejecuta una sola vez y cada k corre únicamente su refinamiento voraz. Medido en N25A: el primer k cuesta 22–120 s según la estrategia y cada k adicional cuesta 0,1–0,2 s. El tiempo reportado por celda es honesto: la preparación se carga al primer k y los siguientes reportan solo su refinamiento.

4.8. Optimizaciones implementadas

- **Tabla T una vez** y reutilizada para todo k (`apply_strategy_for_ks`); a nivel de tabla de evaluación esto divide el cómputo por cuatro.
- **Tabla T vectorizada por niveles** (Algoritmo 2): arreglo indexado por entero de estado en lugar de diccionario de tuplas.
- **Marginal local** $\mathcal{O}(2^{\text{descartadas}})$ en la evaluación de δ_k y en el bucle submodular de QNodes (§4.6).
- **TPM en uint8**: las TPMs deterministas 0/1 se cargan como enteros de un byte (0,84 GB en $n = 25$ frente a 3,36 GB en `float32`); `System` convierte cada columna a `float32` al construir su n-cubo, de modo que toda la aritmética posterior es en punto flotante y el resultado no cambia. Las TPMs continuas conservan `float32` de extremo a extremo.

- **Memoización** de bipartition/marginalize/ marginal_value y del popcount de la tabla.
- **Paralelismo por procesos** (PCD): joblib/loky con SharedMemory, semillas por proceso (SeedSequence) y control de afinidad de hilos (OMP_NUM_THREADS=1) para evitar sobre-suscripción.

Optimizaciones evaluadas y descartadas con datos. Paralelizar el refinamiento voraz no compensa: a $n = 20$ el refinamiento cuesta 0,08–0,2 s frente a 7–8 s de preparación (menos del 2 % del total), y un *pool* de procesos tendría que serializar un subsistema de varios GB. Paralelizar filas de la tabla a $n = 25$ tampoco: un solo worker alcanza 8,4 GB de pico, así que en una máquina de 16 GB no caben dos. Se prototipó además un kernel EMD por lotes con **Numba**, pero el perfilado mostró que la EMD es $< 1\%$ del costo total (lo domina la marginalización), de modo que no compensaba la dependencia ni la compilación JIT: se revirtió y *no* forma parte del proyecto. Las tres decisiones están registradas en la bitácora con las mediciones que las sustentan.

4.9. Metaheurísticas comparativas (GA / SA / Tabú)

Como variante de comparación se implementan tres metaheurísticas —GeneticSIA, AnnealingSIA y TabuSIA (src/controllers/strategies/metaheuristics.py)— que heredan de SIA, delegan en el motor compartido src/funcs/metaheuristic.py y se puntúan con la *misma* pérdida δ_k , de modo que sus resultados son directamente comparables con KGeoMIP/KQNodes y con el óptimo exacto.

Codificación y operadores comunes. Una candidata es un *vector de etiquetas* $\ell \in \{0, \dots, k-1\}^{|F|+|M|}$ que asigna cada átomo (tiempo, índice) a un bloque. La decodificación a KPartition agrupa los átomos por etiqueta; si algún bloque queda vacío la candidata es *infeasible*. La **reparación** restaura la sobreyectividad sobre los k bloques moviendo átomos desde bloques excedentes, garantizando una k -partición estricta (todos los k bloques no vacíos, §2). El evaluador memoiza $\delta_k(\ell)$ por vector canónico para no recomputar la marginalización.

- **Algoritmo genético** [10] (genetic_search). Población de soluciones reparadas; selección por torneo, cruce uniforme por átomo, mutación por reetiquetado con probabilidad p_m y *elitismo* del mejor individuo.
- **Recocido simulado** [8] (simulated_annealing_search). Toma δ_k como energía; en cada paso reetiqueta un átomo y acepta el vecino con probabilidad de Boltzmann $\exp(-\Delta\delta_k/T)$, con enfriamiento geométrico $T \leftarrow \alpha T$.
- **Búsqueda tabú** [9] (tabu_search). Explora un vecindario de reetiquetados, escoge el mejor vecino *no tabú* (con criterio de aspiración si mejora el incumbente) y mantiene una lista tabú de tenencia fija.

Todas son **deterministas** bajo la semilla global (application.numpy_seed) vía np.random.default_rng. Por construcción nunca superan al óptimo exacto ($\delta_k \geq \delta_k^*$); los tests verifican que alcanzan δ_k^* en n pequeño (N3A/N4A) y el análisis empírico (§7) discute su comportamiento al crecer k .

5. Análisis de complejidad

Sea n el número de nodos, m las dimensiones de mecanismo del subsistema y k el número de bloques. Las cotas dominantes verificadas son:

Tabla 1: Complejidad temporal y espacial por componente. r denota las dimensiones descartadas por la marginal local de la candidata evaluada.

Componente	Temporal	Espacial	Cuello de botella
CostTable (build)	$\mathcal{O}(m^2 2^m)$	$\mathcal{O}(n 2^m)$	ancho de banda de memoria
cut pool (KGeoMIP)	$\mathcal{O}(n)$ cortes	$\mathcal{O}(n)$	—
δ_k (una candidata, marginal local)	$\mathcal{O}(n 2^r)$, $r \leq n$	$\mathcal{O}(1)$ extra	promedio del bloque
GREEDY (k bloques)	$\mathcal{O}(k^2 n^2 2^r)$	$\mathcal{O}(2^n)$	evaluaciones δ_k
QNodes (Queyranne)	$\mathcal{O}(n^3 2^r)$ amort.	$\mathcal{O}(2^n)$	medias completas iniciales
Exacto (Stirling)	$\mathcal{O}(S(2n, k) n 2^r)$	$\mathcal{O}(2^n)$	enumeración

Temporal. El costo de KGeoMIP está dominado por la construcción de la tabla, $\mathcal{O}(m^2 2^m)$ con constantes pequeñas (operaciones vectorizadas sobre memoria contigua): 1,6 s a $m = 20$ y ≈ 100 s a $m = 25$ en la máquina de referencia. El de KQNodes está dominado por las $\mathcal{O}(n^3)$ evaluaciones submodulares; con la marginal local cada evaluación cuesta $\mathcal{O}(2^r)$ con r típicamente mucho menor que n (las evaluaciones caras, con $r \approx n$, son pocas y quedan memoizadas), lo que explica el salto empírico de horas proyectadas a ≈ 25 s en N25A. Frente a la fuerza bruta $\mathcal{O}(S(2n, k) \cdot)$, ambas heurísticas reducen el factor combinatorio $S(2n, k)$ a uno polinómico en n y k .

Espacial. Dos términos del mismo orden conviven en $n = 25$: los n -cubos del subsistema ($n 2^n$ valores float32, 3,36 GB) y la tabla T ($2^m \times n$, otros 3,36 GB cuando $m = n$). Con la TPM en uint8 (0,84 GB) el pico medido de KGeoMIP en N25A es de 8,4 GB, dentro de una máquina de 16 GB; KQNodes, que no construye T , alcanza 5,8 GB.

Análisis de casos. El **mejor caso** del voraz ocurre cuando el primer corte ya aísla la masa de probabilidad (pérdida casi nula, p. ej. N15A con $\delta_k = 0,027$). El **peor caso** aparece cuando ningún corte del pool reduce δ_k y deben probarse todos los pares (bloque, corte) en los $k - 1$ niveles. Para la marginal local el peor caso por candidata es un bloque que descarta casi todas las dimensiones ($r \approx n$, un promedio de 2^{24} valores ≈ 35 ms a $n = 25$); esos casos coinciden con los lados pequeños de las particiones, se repiten entre candidatas y por eso la memoización los absorbe.

Validación empírica de las cotas. La predicción $\mathcal{O}(m^2 2^m)$ del build se contrasta con los tiempos medidos: de $m = 20$ a $m = 25$ la cota predice un factor $\approx 2^5 \cdot (25/20)^2 = 50$ y el medido es $100/1,6 \approx 61$ (el exceso es presión de caché y de memoria, coherente con un algoritmo limitado por ancho de banda). Para KQNodes, el tiempo total en el subsistema completo creció de 1,6 s ($n = 20$) a 25,2 s ($n = 25$), factor ≈ 16 : muy por encima del ≈ 2 que predeciría n^3 aislado, porque el costo 2^r de la marginal local crece con n y domina a esta escala.

6. Detalles de implementación

6.1. Métodos principales

- `KGeoMIP(tpm, initial_state, k)` y `KQNodes(tpm, initial_state, k)` — construyen la estrategia para un k dado.
- `apply_strategy(condition, purview, mechanism) -> Solution` — prepara el subsistema, construye el *cut pool* y delega en `greedy_k_partition`; devuelve un `Solution` (estrategia, pérdida, partición, tiempo).

- `System.k_partition_marginal_distribution(partition)` — evalúa la distribución reconstruida de una `KPartition` validada con la marginal local (§4.6), sin materializar el tensor completo.
- `delta_k(subsystem, partition, baseline)` → (float, ndarray) — pérdida δ_k y distribución particionada.
- `KPartition.from_blocks(blocks, future_universe, present_universe)` — fábrica que valida y normaliza los bloques.

```

1 def delta_k(subsystem, partition, baseline_distribution=None):
2     original = baseline_distribution
3     if original is None:
4         original = subsystem.marginal_distribution()
5     partitioned = subsystem.k_partition_marginal_distribution(partition)
6     return effect_emd(original, partitioned), partitioned

```

Listing 1: Núcleo de la evaluación δ_k (src/funcs/emd.py).

6.2. Dependencias externas

Tabla 2: Dependencias principales y su uso. Todas son de núcleo y se instalan con `uv sync`; el proyecto no declara extras opcionales.

Biblioteca	Versión	Uso
numpy	2.4.6	tensores, vectorización, EMD analítica
scipy	1.17.1	clustering espectral (csgraph, linalg)
pandas	3.0.3	E/S de TPM y tabla de resultados
more_itertools	11.1.0	particiones de Stirling (exacto)
matplotlib	3.10.9	figuras y visualización de particiones
pyphi	1.2.0	validación cruzada [3] (6–10 nodos)
joblib / psutil	—	PCD (procesos), medición
pyemd	2.0.0	EMD causal (métrica Hamming) de comprobación
plotly	6.8.0	figuras interactivas
streamlit	1.58.0	interfaz web

6.3. Ingeniería de software

- **Validación de entradas:** `KPartition` verifica cobertura, disyunción y no-vacuidad estricta; `apply_strategy` exige $k \geq 2$.
- **Manejo de errores:** `ValueError` (parámetros/partición inválida) y `RuntimeError` (pool incapaz de refinar a k bloques) con mensajes descriptivos.
- **Logging:** `SafeLogger` (carpeta logs/); profiling con `@profile` (HTML en review/profiling/).
- **Reproducibilidad:** semilla central en `application`; semillas derivadas por proceso con `SeedSequence.spawn`.

6.4. Tests implementados

El conjunto de pruebas (`uv run pytest, 275`) cubre: regresión $k = 2$ ($K\text{GeoMIP} \equiv \text{GeoMIP}$, $KQ\text{Nodes} \equiv Q\text{Nodes}$), validación $k \in \{3, 4, 5\}$ contra el exacto, semántica estricta de `KPartition`, determinismo del clustering y de

las **metaheurísticas** (misma semilla $\Rightarrow$ misma partición), cota $\delta_k \geq \delta_k^*$ de las siete estrategias, integración *end-to-end* (cargar $\rightarrow$ estrategia $\rightarrow$ Solution e ida y vuelta generar/cargar), EMD causal opcional (importorskip), figuras interactivas, registro sin interfaz (*headless*) (runner), precisión float32 vs float64 ($|\Delta| \leq 10^{-5}$), métricas y PCD. Triage histórico de QNodes congelado en `tests/unit/test_qnodes_triage.py`.

7. Resultados experimentales

7.1. Datasets

TPMs deterministas `N{n}-{letra}.csv` (2^n filas $\times$ n columnas, little-endian), generadas con semilla 73. Hay dos conjuntos de resultados:

- **Tabla de evaluación oficial** (`data/results/resultados.xlsx`): cinco redes (N10A, N15B, N20A, N22A, N25A) con ≈ 50 subsistemas cada una y $k \in \{2, 3, 4, 5\}$, resuelta por las dos estrategias núcleo, KGeoMIP y KQNodes. Está llena al 100 % y cada celda guarda la partición, su δ_k real y su tiempo.
- **Benchmark comparativo** (`data/results/benchmark.csv`): las *siete* estrategias (KGeoMIP, KQNodes, Clustering $\times 2$, GA/SA/Tabú) sobre N10A, N15A y N20A, usado para comparar precisión y tiempo entre variantes.

Las tablas siguientes muestran una fila representativa: el subsistema completo (*purview* y *mechanism* con todos los nodos), que es el caso más caro de cada red.

7.2. Métricas

- **Pérdida** δ_k (Ec. (1)): menor es mejor.
- **Tiempo de ejecución** (s, perfilador apagado).
- **Tasa de acierto exacto**: fracción de casos donde δ_k iguala al óptimo exacto (n pequeño).
- **Error relativo de Φ y distancia de Jaccard** entre particiones (conteo de pares co-asignados).
- **Aceleración** (*speedup*) y **pendiente de escalabilidad** (log–log).

7.3. Tabla de resultados

La tabla 3 confirma que las dos estrategias núcleo coinciden en δ_k sobre el subsistema completo de cada red, incluidas N20A, N22A y N25A. Las pequeñas diferencias en $k = 5$ (p. ej. N10A: 1,9180 vs 1,9277) reflejan que ambas son voraces y a veces eligen cortes distintos de calidad casi igual.

Tabla 3: Pérdida δ_k de las dos estrategias núcleo en el subsistema completo (tabla oficial, `resultados.xlsx`).

Red	δ_k ($k = 2$)		δ_k ($k = 5$)	
	KGeoMIP	KQNodes	KGeoMIP	KQNodes
N10A	0,4727	0,4727	1,9180	1,9277
N15B	0,0000	0,0000	0,0000	0,0000
N20A	0,4991	0,4991	1,9970	1,9970
N22A	0,4996	0,4996	1,9990	1,9990
N25A	0,4998	0,4998	1,9994	1,9994

N15B es una red determinista con partición perfecta ($\delta_k = 0$). Las filas con subsistema más pequeño dan pérdidas menores; aquí se muestra el caso más exigente de cada red. Estado inicial 1000...0 (un nodo activo), la convención de la tabla oficial `resultados.xlsx`; por eso estos valores no son directamente comparables con los de la Tabla 4, que evalúa otro subsistema (estado 1...1).

Tabla 4: Metaheurísticas vs. la mejor pérdida convergente de las estrategias núcleo. La columna “Mejor” es el menor δ_k alcanzado por las búsquedas independientes, *no* el óptimo global (intratable a este n); coincide con KGeoMIP en estas filas. Estado inicial 1...1 y red N15A, distintos del subsistema de la Tabla 3 (estado 1000...0, red N15B), de modo que las dos tablas no son comparables celda a celda.

Red	k	Mejor (conv.)	Tabú	Genética	Recocido
N10A	2	0,4697	0,4697	0,4697	0,4727
N10A	3	0,9424	0,9424	0,9580	0,9639
N10A	4	1,4229	1,4229	2,3916	1,4365
N15A	2	0,0268	0,0268	0,0270	0,0271
N15A	3	0,0538	0,0538	0,0539	0,0540
N15A	4	0,0809	0,0809	0,8331	0,8346

Tabla 5: Tiempo de ejecución (s) en el subsistema completo, perfilador apagado. La preparación compartida (tabla de costos y *cut pool*) se carga a $k = 2$; los $k = 3, 4, 5$ solo refinan.

Red	KGeoMIP $k=2$	KGeoMIP $k \geq 3$	KQNodes $k=2$	KQNodes $k \geq 3$
N10A	0,01	0,01–0,02	0,07	0,01–0,03
N15B	0,04	0,03–0,06	0,32	0,04–0,08
N20A	1,64	0,05–0,10	1,61	0,07–0,12
N22A	8,92	0,07–0,13	3,99	0,09–0,16
N25A	100,35	0,09–0,17	25,21	0,12–0,22

Medido en un Intel Core i7-10510U (8 hilos), 16 GB, en corridas aisladas y con el perfilador apagado. A $n = 25$ la preparación de KGeoMIP es el coste dominante (≈ 101 s para los cuatro k); KQNodes resuelve los cuatro en ≈ 26 s.

7.4. Gráficas y visualizaciones

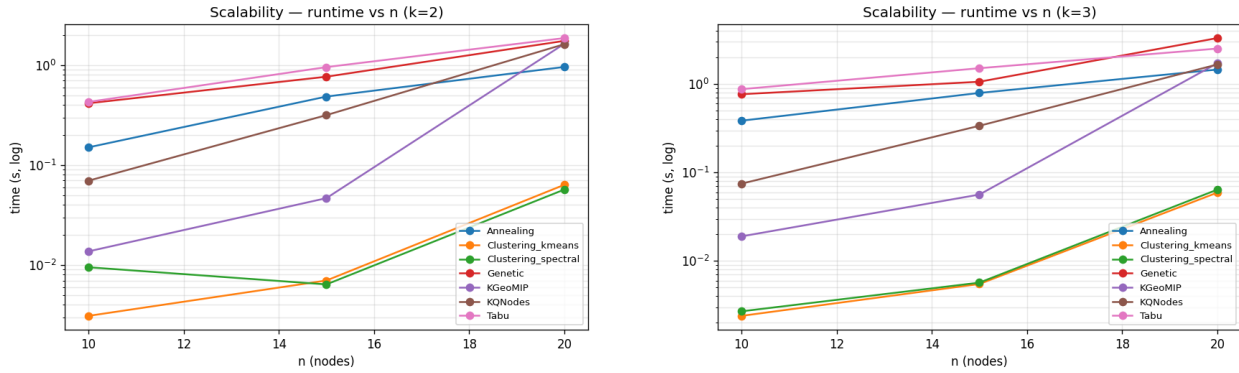


Figura 5: Escalabilidad (tiempo vs n , escala log) para $k = 2$ (izq.) y $k = 3$ (der.) sobre el benchmark N10A/N15A/N20A. KGeoMIP es varias veces más rápido que KQNodes a n pequeño ($\sim 5\text{--}7\times$) y ambas convergen hacia $n = 20$; a $n \geq 22$ la relación se invierte (Tabla 5).

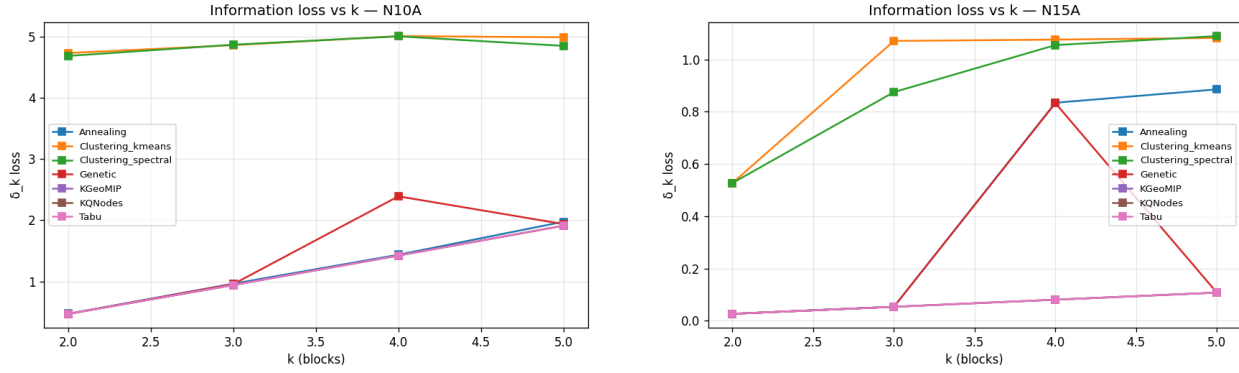


Figura 6: Pérdida δ_k vs k en N10A (izq.) y N15A (der.): monótona creciente, como exige la semántica estricta.

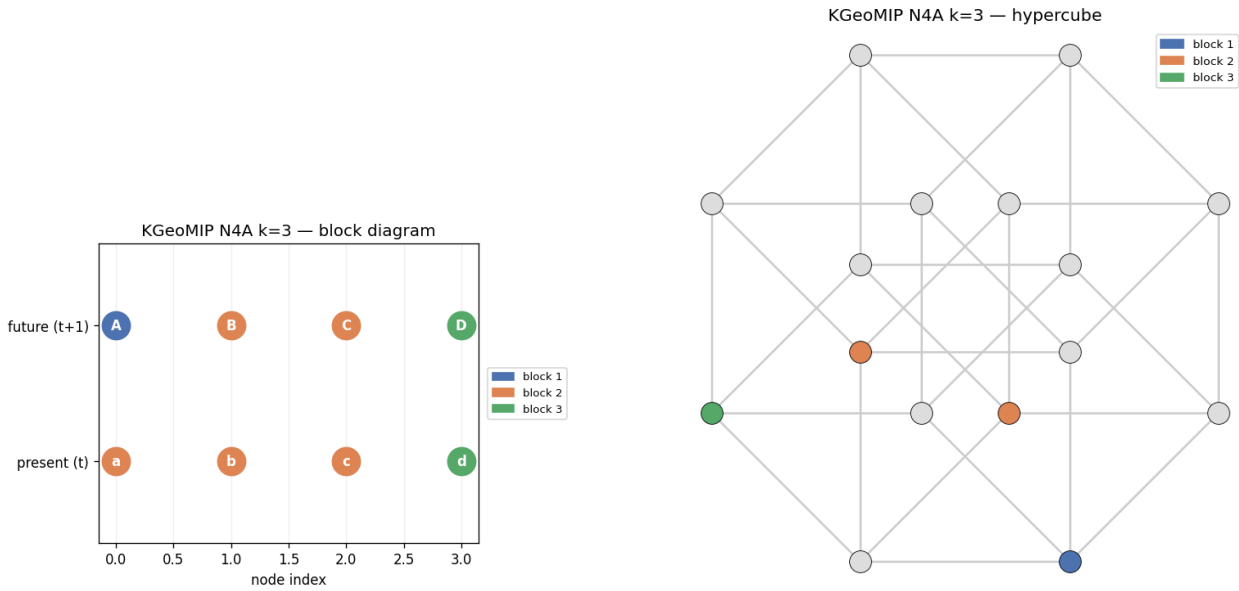


Figura 7: k -partición hallada (N4A, $k = 3$): diagrama de bloques presente/futuro (izq.) y proyección del hipercubo de nodos (der.).

7.5. Visualización interactiva e interfaz web

Las mismas vistas se ofrecen de forma **interactiva** (Plotly) mediante `src/viz/interactive.py`: diagrama de bloques de la partición con *hover* por bloque, δ_k vs k y escalabilidad con ejes navegables. `scripts/make_interactive.py` las exporta como HTML autocontenido (plotly.js por CDN). Todo se integra en una **interfaz web** (Streamlit, `streamlit_app.py`) que permite, sin línea de comandos: generar una TPM desde el navegador, elegir estrategia/ k /máscaras, y obtener la partición, su δ_k , la distribución marginal y las figuras estática e interactiva. El registro sin interfaz (*headless*) `src/funcs/runner.py` (`run_analysis`) es el núcleo compartido por la UI y los scripts. La interfaz web vive en `streamlit_app.py`.

7.6. Análisis de resultados

- **KGeoMIP $\equiv$ KQNodes en calidad:** ambas hallan la misma pérdida (diferencias $\leq 10^{-4}$ por desempates), confirmando que comparten el motor voraz y la métrica δ_k .

- **Núcleo $\gg$ línea base:** las estrategias núcleo superan al clustering por $\sim 10\times$ en N10A y $\sim 20\times$ en N15A; el clustering optimiza una métrica de grafo, no δ_k .
- **Costo relativo según la escala:** en redes pequeñas KQNodes es $\sim 7\times$ más lento que KGeoMIP por la búsqueda Queyranne, pero a $n = 22$ y $n = 25$ la relación se invierte: KQNodes resuelve el subsistema completo en ≈ 4 s y ≈ 25 s frente a ≈ 9 s y ≈ 100 s de KGeoMIP (Tabla 5), porque su preparación es más barata que la tabla de costos geométrica a esos tamaños.
- **Monotonía:** δ_k crece con k (Fig. 6), coherente con que más bloques destruyen más información.
- **Ambas escalan a $n = 25$:** KGeoMIP y KQNodes llenan la tabla oficial completa hasta N25A puntuando cada celda con δ_k (Tabla 3). El salto de tiempo de $n = 20$ a $n = 25$ valida empíricamente la cota $\mathcal{O}(2^n)$ de la preparación; el límite práctico es la memoria, no el tiempo (§8).
- **Metaheurísticas (Tabla 4):** la *búsqueda tabú* iguala la mejor pérdida convergente en toda la tabla; el *recocido* se le aproxima salvo en k alto; la *genética* acierta en $k = 2$ pero se degrada al crecer k , pues el espacio k^{2n} excede el presupuesto de ~ 1200 evaluaciones por defecto. Es el comportamiento esperado de una metaheurística sin reinicio y refuerza que las estrategias núcleo (deterministas) son la mejor opción aquí.

7.7. Interpretación de los resultados

Más allá de los números, conviene leer qué significan para la teoría y para el uso del sistema.

Qué dice la pérdida sobre el sistema. Una pérdida δ_k baja indica que el sistema se puede romper en k bloques perdiendo poca información integrada: sus partes son casi independientes en su efecto causal. Una pérdida alta indica lo contrario, que el todo depende fuertemente de la interacción entre bloques. N15B es el caso extremo: $\delta_k = 0$ para todo k , así que esa red determinista es separable sin pérdida. En las demás redes la pérdida crece de forma ordenada con k (cada bloque adicional rompe más estructura), lo que es la firma esperada de una semántica estricta de partición.

Por qué las dos estrategias núcleo coinciden. KGeoMIP parte de cortes geométricos guiados por la tabla de costos y KQNodes de una secuencia submodular tipo Queyranne, pero ambas refinan con la misma pérdida δ_k y, sobre el subsistema completo de cada red, llegan al mismo valor. Que dos búsquedas con orígenes distintos converjan al mismo mínimo es la evidencia práctica de optimalidad que se puede dar a esta escala, donde enumerar todas las k -particiones es inviable. Las metaheurísticas independientes (sobre todo Tabú) refuerzan esa coincidencia.

Cuándo usar cada una. En redes pequeñas las dos son intercambiables y muy rápidas. A $n = 22$ y $n = 25$ KQNodes resuelve el subsistema completo más rápido que KGeoMIP (su preparación es más barata que la tabla de costos geométrica), así que es la opción preferible cuando el tiempo importa. KGeoMIP sigue siendo útil porque su *cut pool* geométrico aporta diversidad de cortes (a veces halla una partición distinta de igual δ_k) y es más rápida en n pequeño; en la franja verificable por fuerza bruta ($n \leq 6$) ambas tienen una tasa de acierto exacto equivalente. La recomendación operativa es ejecutar ambas y tomar el mínimo.

Dónde está el límite. El techo de $n \approx 25$ no es de tiempo sino de memoria: la tabla de costos de KGeoMIP a subsistema completo de $n = 25$ ronda los 8,4 GB. En una máquina de 16 GB eso obliga a ejecutar las filas grandes en serie, que es lo que fija el tiempo total de llenar la tabla oficial. Subir de 25 pediría o más memoria o un esquema que evite materializar la tabla completa.

7.8. Validación de correctitud

La validación se ejecuta con `uv run scripts/validate.py`, que tiene tres subcomandos: `correctness`, `optimality` y `external`.

7.8.1. Frente al óptimo exacto y al legado $k = 2$

- **Regresión $k = 2$:** KGeoMIP y KQNodes reproducen exactamente GeoMIP y QNodes legados.
- **Oráculo:** sobre los casos golden, el núcleo (`BruteForce` $\equiv$ GeoMIP) coincide 21/21; en $k = 2$ KGeoMIP iguala el óptimo exacto y KQNodes acierta 9/10. Para $k \geq 3$ ambas son voraces y pueden quedar por encima del exacto (§7.7).
- **Consistencia:** $\text{loss} = \delta_k$ y $\text{loss} \geq$ óptimo exacto en 18/18 casos del verificador; float32 igual a float64 dentro de 10^{-5} .

7.8.2. Frente al proyecto original de la docente

La única referencia externa de la que se reporta es el proyecto original entregado por la docente (`.core/core_00`), cuyo repositorio es github.com/JuManoel/proyecto-analisis-20261. El subcomando `external` comprueba dos cosas y mide los resultados:

- **TPM idéntica:** el archivo `data/samples/N15A.csv` del que muestreamos es idéntico byte a byte al que trae el proyecto original.
- **Reproducción de GeoMIP:** las filas de su `resultados_Geometric.xlsx` (N15A, TPM continua) se reproducen con nuestro `GeometricSIA` dentro de la tolerancia float32. Tres filas medidas: 0,0002500843 vs 0,0002501011; 0,0002678838 vs 0,0002678633 (dos veces). Las seis primeras filas reproducen con OK.

7.8.3. Re-evaluación de las particiones almacenadas

Cada celda de la tabla oficial guarda la partición, su pérdida y su tiempo. El subcomando `external-verify-partitions` vuelve a leer cada partición, la valida como k -partición estricta (cobertura y disjunción de la sección 2.2) y la re-puntúa con la δ_k oficial sobre el subsistema reconstruido. Sobre las cinco hojas de la tabla (N10A, N15B, N20A, N22A, N25A), las 1992 celdas llenas dan 0 particiones inválidas y 0 desajustes de pérdida: las particiones reportadas son las que realmente se evaluaron, no solo sus números.

8. Limitaciones y trabajo futuro

8.1. Limitaciones conocidas

- **Techo de escala $n \approx 25$:** KGeoMIP y KQNodes llenan la tabla oficial completa hasta N25A puntuando cada celda con δ_k . El límite es la memoria: la tabla de costos de KGeoMIP a subsistema completo de $n = 25$ tiene un pico de $\approx 8,4$ GB, así que en una máquina de 16 GB las filas grandes se ejecutan en serie. No se promete escalar más allá de $n \approx 25$.
- **Tiempos a $n = 25$:** sobre el subsistema completo de N25A (la fila más cara), KGeoMIP resuelve $k = 2..5$ en ≈ 101 s y KQNodes en ≈ 26 s. El grueso es la preparación compartida (tabla de costos y *cut pool*), que se carga a $k = 2$; los $k = 3, 4, 5$ solo refinan y tardan décimas de segundo. Las filas con subsistema más pequeño son mucho más baratas.

- **Optimalidad:** el motor voraz es heurístico; no garantiza la k -MIP global (sí factibilidad, monotonía y validación contra el exacto en n pequeño y convergencia de estrategias independientes a mayor escala).

8.2. Supuestos y restricciones

TPM determinista binaria; independencia condicional del repertorio de efecto (que habilita la EMD-efecto analítica $\mathcal{O}(n)$); GIL activo en Python 3.14.5, por lo que el paralelismo es por procesos, no por hilos.

8.3. Mejoras potenciales y trabajo futuro

La marginalización local, la carga en uint8 y las metaheurísticas comparativas ya están implementadas (secciones 4.6 y 4.9). Lo que queda como trabajo futuro:

- Llenado por lotes consciente de la memoria: paralelizar por procesos solo las hojas que caben en RAM (hasta $n \approx 20$) y dejar las filas grandes en serie, para recortar el tiempo de la tabla completa sin arriesgar OOM.
- Refinamiento con anticipación (*look-ahead*) o búsqueda en haz (*beam search*) para acercarse al óptimo global manteniendo costo polinómico.
- Aceleración GPU solo si el volumen justifica la transferencia H2D (analizado: no rentable para los tamaños actuales).

9. Apéndices técnicos

9.1. Reducción $k = 2 \Rightarrow$ bipartición legada

Proposición. Para $k = 2$, GREEDYKPARTITION ejecuta exactamente un refinamiento y el resultado coincide con la bipartición de la estrategia base.

Bosquejo. El bucle while corre mientras $|blocks| < 2$, es decir una sola iteración partiendo de $[(F, M)]$. El único bloque es el universo completo, de modo que dividirlo con un corte c produce $(c, \bar{c})$, que es la bipartición candidata original. Como el pool y la métrica δ_k son los del método base, se selecciona la misma bipartición. $\square$

9.2. Reproducción de figuras y diagramas

```

1 # Tabla y metrics
2 uv run scripts/run_benchmark.py
3 uv run scripts/validate.py correctness
4 # Figuras (matplotlib) y visualizacion de particiones
5 uv run scripts/make_figures.py
6 uv run scripts/make_viz.py --net N4A --k 3 --strategy KGeoMIP
7 # Diagramas UML editables (PlantUML)
8 plantuml docs/manuales/diagrams/*.puml

```

Listing 2: Comandos de reproducción.

9.3. Uso de inteligencia artificial generativa

Durante el desarrollo se utilizó **Claude Code (Opus 4.8)** como asistente, documentado entrada por entrada en logs/ai_agent_changelog.md (con el *prompt* del usuario, parámetros reales y decisiones).

Etapas y ejemplos.

- **Diseño:** extracción del motor voraz compartido (`greedy_k_partition`) y decisión de semántica estricta tras medir que la semántica débil degeneraba $k > 2$ a bipartición.
- **Implementación:** vectorización de `CostTable`, conversión a `float32`, infraestructura PCD (`parallel.py`) y visualización (`src/viz`).
- **Depuración/optimización:** perfilado con `pyinstrument`; hallazgo de que Numba en lotes pequeños era $\sim 70\%$ más lento (revertido) y de que la marginalización domina el costo.
- **Documentación:** estructuración de manuales, diagramas y tablas.

Reflexión crítica. La IA aceleró tareas mecánicas (vectorización, código repetitivo, redacción) y propuso refactorizaciones útiles; sin embargo, varias afirmaciones de evaluación automática se basaron en código obsoleto y debieron *verificarse* contra el repositorio real (regla “verificar, no asumir”). El criterio de validación contra oráculo y micro-mediciones (*microbenchmarks*) aisladas fue imprescindible para no aceptar mejoras ilusorias. La comprensión profunda del algoritmo y la responsabilidad de las decisiones permanecen en el equipo.

9.4. Referencias

Referencias

- [1] G. Tononi. *An information integration theory of consciousness*. BMC Neuroscience, 5:42, 2004.
- [2] M. Oizumi, L. Albantakis, G. Tononi. *From the phenomenology to the mechanisms of consciousness: Integrated Information Theory 3.0*. PLoS Computational Biology, 10(5):e1003588, 2014.
- [3] W. Mayner et al. *PyPhi: A toolbox for integrated information theory*. PLoS Computational Biology, 14(7):e1006343, 2018.
- [4] M. Queyranne. *Minimizing symmetric submodular functions*. Mathematical Programming, 82:3–12, 1998.
- [5] G. Nemhauser, L. Wolsey, M. Fisher. *An analysis of approximations for maximizing submodular set functions—I*. Mathematical Programming, 14:265–294, 1978.
- [6] Y. Rubner, C. Tomasi, L. Guibas. *The Earth Mover’s Distance as a metric for image retrieval*. International Journal of Computer Vision, 40(2):99–121, 2000.
- [7] U. von Luxburg. *A tutorial on spectral clustering*. Statistics and Computing, 17(4):395–416, 2007.
- [8] S. Kirkpatrick, C. Gelatt, M. Vecchi. *Optimization by simulated annealing*. Science, 220(4598):671–680, 1983.
- [9] F. Glover. *Future paths for integer programming and links to artificial intelligence*. Computers & Operations Research, 13(5):533–549, 1986.
- [10] J. Holland. *Adaptation in Natural and Artificial Systems*. University of Michigan Press, 1975.
- [11] Universidad de Caldas. *Proyecto K-QGMIP — Especificación* (docs/Proyecto_KQMIP.md), 2026-1.